

Yesterday 3:13 PM

Coding questions for stream and lambda

Find duplicate elements.

Find first non-repeating character.

Find frequency of characters.

Find the second-highest salary.

Find employees having salary greater than average.

Group employees by department.

Find highest-paid employee per department.

Partition employees based on salary.

Convert List<Employee> to Map<id, Employee>.

Find duplicate employees based on ID.

Show more ▾

Absolutely. For a 5-year Java developer interview, these are very common **Stream API + Lambda coding questions**. Below I'll give you the **interview-ready approach**, code, and the important points you should explain.

Assume this common `Employee` class:

 **Java**



```
class Employee {  
    private int id;  
    private String name;  
    private String department;  
    private double salary;  
  
    // constructor, getters  
}
```

1. Find duplicate elements in a List

Question

</> Java



```
List<Integer> numbers = Arrays.asList(1, 2, 3, 2, 4, 5, 1);
```

Answer

</> Java



```
Set<Integer> seen = new HashSet<>();

List<Integer> duplicates = numbers.stream()
    .filter(n -> !seen.add(n))
    .distinct()
    .collect(Collectors.toList());

System.out.println(duplicates);
```

Output:

```
[2, 1]
```



How to explain

Set.add() returns:

- true → element is being added for the first time
- false → element already exists

Therefore:

</> Java



```
!seen.add(n)
```

identifies duplicates.

2. Find first non-repeating character

Question

```
"swiss"
```



Expected:

w



Answer

</> Java



```
String str = "swiss";

Character result = str.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(
        Function.identity(),
        LinkedHashMap::new,
        Collectors.counting()
    ))
    .entrySet()
    .stream()
    .filter(entry -> entry.getValue() == 1)
    .map(Map.Entry::getKey)
    .findFirst()
    .orElse(null);

System.out.println(result);
```

Important interview point

Why LinkedHashMap ?

Because we need to preserve the **insertion order of characters**.

If you use normal HashMap , the first non-repeating character cannot reliably be determined based on original order.

3. Find frequency of characters

Question

"hello"



Expected:

h=1
e=1
l=2
o=1



Answer

</> Java



```
Map<Character, Long> frequency =  
    str.chars()  
        .mapToObj(c -> (char) c)  
        .collect(Collectors.groupingBy(  
            Function.identity(),  
            Collectors.counting()  
        ));  
  
System.out.println(frequency);
```

Core concept

</> Java



```
groupingBy(...)
```

groups elements, while:

</> Java



```
counting()
```

counts each group.

4. Find second-highest salary

</> Java



```
Optional<Double> secondHighest =  
    employees.stream()  
        .map(Employee::getSalary)  
        .distinct()  
        .sorted(Comparator.reverseOrder())
```

```
.skip(1)
.findFirst();
```

If you want the **Employee**:

</> Java



```
Optional<Employee> secondHighestEmployee =
    employees.stream()
        .sorted(Comparator.comparingDouble(Employee::getSalary)
            .reversed())
        .skip(1)
        .findFirst();
```

Important difference

The first version:

</> Java



```
.map(Employee::getSalary)
.distinct()
```

finds the **second-highest distinct salary**.

Example:

```
100
100
90
80
```



Result:

```
90
```



5. Find employees having salary greater than average

First calculate average:

</> Java



```
double averageSalary = employees.stream()
    .mapToDouble(Employee::getSalary)
    .average()
    .orElse(0);
```

Then:

```
</> Java

List<Employee> result = employees.stream()
    .filter(e -> e.getSalary() > averageSalary)
    .collect(Collectors.toList());
```

Interview explanation

I use:

```
</> Java

mapToDouble()
```

because salary is numeric and it gives access to:

```
</> Java

average()
sum()
max()
min()
```

without unnecessary boxing.

6. Group employees by department

Question

Group employees based on department.

```
</> Java

Map<String, List<Employee>> employeesByDepartment =
    employees.stream()
        .collect(Collectors.groupingBy(
```

```
Employee::getDepartment  
));
```

Result conceptually:

```
IT      -> [John, Alex]  
HR      -> [David, Mike]  
Finance -> [Robert]
```



This is one of the **most important Stream API** interview questions.

7. Find highest-paid employee per department

This is a very common 5-year interview question.

</> Java



```
Map<String, Optional<Employee>> result =  
    employees.stream()  
        .collect(Collectors.groupingBy(  
            Employee::getDepartment,  
            Collectors.maxBy(  
                Comparator.comparingDouble(Employee::getSalary)  
            )  
        ));
```

You get:

```
IT      -> Optional<Employee>  
HR      -> Optional<Employee>  
Finance -> Optional<Employee>
```



If you want `Map<String, Employee>` instead:

</> Java



```
Map<String, Employee> result =  
    employees.stream()  
        .collect(Collectors.groupingBy(  
            Employee::getDepartment,  
            Collectors.collectingAndThen(  
                Collectors.maxBy(  
                    Comparator.comparingDouble(Employee::getSalary)  
                ),  
            )  
        ));
```

```
Optional::get  
    )  
    ));
```

Better interview answer

I'd normally prefer the first version because `Optional` safely represents the possibility that a group has no employee.

8. Partition employees based on salary

Question

Separate employees into:

```
salary > 100000  
salary <= 100000
```



Answer

```
</> Java  
  
Map<Boolean, List<Employee>> result =  
    employees.stream()  
        .collect(Collectors.partitioningBy(  
            e -> e.getSalary() > 100000  
        ));
```



Result:

```
true -> employees with salary > 100000  
false -> employees with salary <= 100000
```



`groupingBy()` vs `partitioningBy()`

This is an excellent follow-up question.

Use:

```
</> Java
```



```
groupingBy()
```

when you have **multiple categories**.

Use:

```
</> Java
```



```
partitioningBy()
```

when you have a **boolean condition**.

9. Convert List<Employee> to Map<id, Employee>

```
</> Java
```



```
Map<Integer, Employee> employeeMap =  
    employees.stream()  
        .collect(Collectors.toMap(  
            Employee::getId,  
            Function.identity()  
        ));
```

Potential problem

What if two employees have the same ID?

You'll get:

```
IllegalStateException: Duplicate key
```



So in production code, handle duplicates:

```
</> Java
```



```
Map<Integer, Employee> employeeMap =  
    employees.stream()  
        .collect(Collectors.toMap(  
            Employee::getId,  
            Function.identity(),  
            (existing, replacement) -> existing  
        ));
```

This means:

duplicate ID → keep existing employee



10. Find duplicate employees based on ID

</> Java



```
Set<Integer> seen = new HashSet<>();

List<Employee> duplicates =
    employees.stream()
        .filter(e -> !seen.add(e.getId()))
        .collect(Collectors.toList());
```

If you want each duplicate ID only once:

</> Java



```
Set<Integer> seen = new HashSet<>();

List<Employee> duplicates =
    employees.stream()
        .filter(e -> !seen.add(e.getId()))
        .distinct()
        .collect(Collectors.toList());
```

Better approach when you only need duplicate IDs

</> Java



```
Set<Integer> duplicateIds =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getId,
            Collectors.counting()
        ))
        .entrySet()
        .stream()
        .filter(e -> e.getValue() > 1)
        .map(Map.Entry::getKey)
        .collect(Collectors.toSet());
```

11. Flatten List<List<Integer>>

Input

</> Java



```
List<List<Integer>> numbers = Arrays.asList(  
    Arrays.asList(1, 2, 3),  
    Arrays.asList(4, 5),  
    Arrays.asList(6, 7)  
);
```

Answer

</> Java



```
List<Integer> result =  
    numbers.stream()  
        .flatMap(List::stream)  
        .collect(Collectors.toList());
```

Output:

[1, 2, 3, 4, 5, 6, 7]



Very important interview concept

map() :

</> Java



```
stream.map(List::stream)
```

produces:

Stream<Stream<Integer>>



Whereas:

</> Java



```
flatMap(List::stream)
```

produces:

```
Stream<Integer>
```

`flatMap()` = **transform + flatten**.

12. Find top 3 salaries

`</>` Java

```
List<Double> top3 =  
    employees.stream()  
        .map(Employee::getSalary)  
        .distinct()  
        .sorted(Comparator.reverseOrder())  
        .limit(3)  
        .collect(Collectors.toList());
```

Example:

```
100000  
90000  
80000  
70000  
60000
```

Result:

```
[100000, 90000, 80000]
```

Why `distinct()` ?

Suppose salaries are:

```
100000  
100000  
90000  
80000
```

Without `distinct()` :

[100000, 100000, 90000]



With `distinct()`:

[100000, 90000, 80000]



So clarify whether interviewer means **top 3 employees** or **top 3 distinct salaries**.

13. Find top 3 highest-paid employees

`</>` Java



```
List<Employee> top3 =  
    employees.stream()  
        .sorted(Comparator.comparingDouble(Employee::getSalary)  
            .reversed())  
        .limit(3)  
        .collect(Collectors.toList());
```

If salaries are:

A -> 100000
B -> 100000
C -> 90000
D -> 80000



Result:

A
B
C



14. Sort employees by multiple fields

Requirement

Sort by:

1. Department ascending
2. Salary descending

3. Name ascending

Answer

</> Java



```
List<Employee> sorted =  
    employees.stream()  
        .sorted(  
            Comparator.comparing(Employee::getDepartment)  
                .thenComparing(  
                    Comparator.comparingDouble(Employee::getSalary)  
                        .reversed()  
                )  
            .thenComparing(Employee::getName)  
        )  
    .collect(Collectors.toList());
```

This is a very important pattern:

</> Java



```
Comparator.comparing(...)  
    .thenComparing(...)  
    .thenComparing(...)
```

★ Important interview variations

For a 5-year Java interview, don't just memorize the above. Interviewers commonly modify the requirement.

Example 1 — Highest salary

</> Java



```
employees.stream()  
    .max(Comparator.comparingDouble(Employee::getSalary));
```

Example 2 — Lowest salary

</> Java



```
employees.stream()  
    .min(Comparator.comparingDouble(Employee::getSalary));
```

Example 3 — Average salary

</> Java



```
double avg = employees.stream()  
    .mapToDouble(Employee::getSalary)  
    .average()  
    .orElse(0);
```

Example 4 — Total salary

</> Java



```
double total = employees.stream()  
    .mapToDouble(Employee::getSalary)  
    .sum();
```

Example 5 — Count employees per department

</> Java



```
Map<String, Long> count =  
    employees.stream()  
        .collect(Collectors.groupingBy(  
            Employee::getDepartment,  
            Collectors.counting()  
        ));
```

Example 6 — Average salary per department

</> Java



```
Map<String, Double> avgSalary =  
    employees.stream()  
        .collect(Collectors.groupingBy(  
            Employee::getDepartment,  
            Collectors.averagingDouble(Employee::getSalary)  
        ));
```

Example 7 — Highest salary per department

</> Java



```
Map<String, Optional<Employee>> highest =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.maxBy(
                Comparator.comparingDouble(Employee::getSalary)
            )
        ));
```



The patterns you should memorize

Most Stream coding questions are combinations of these:

Requirement	Stream operation
Transform	map()
Flatten	flatMap()
Filter	filter()
Remove duplicates	distinct()
Sort	sorted()
First N	limit()
Skip N	skip()
Group	groupingBy()
Split into true/false	partitioningBy()
Count	count() / Collectors.counting()
Maximum	max() / maxBy()
Minimum	min() / minBy()
Sum	sum()

Requirement	Stream operation
Average	<code>average()</code>
Convert to Map	<code>toMap()</code>
Combine values	<code>reduce()</code>
Find first	<code>findFirst()</code>
Find any	<code>findAny()</code>

The biggest interview trick

Don't start writing random stream operations.

First identify the **shape of the problem**:

Filter?
↓
Transform?
↓
Group?
↓
Sort?
↓
Aggregate?
↓
Collect?



For example:

"Find highest-paid employee in each department"

Think:

Employees
↓
`groupingBy(department)`
↓
`maxBy(salary)`
↓
`Map<Department, Employee>`



That's the level of thinking I'd expect from a **5-year Java candidate**.



Find the employee with maximum salary in each department.

Count occurrences of each word.

Find common elements between two lists.

Convert a list into comma-separated String.

These are also very common **Java 8+ Stream/Lambda interview questions**. I'll show the clean interview answer plus what to explain.

1. Find employee with maximum salary in each department

Input

`</> Java`



```
List<Employee> employees;
```

Solution 1 — Using `groupBy()` + `maxBy()`

`</> Java`



```
Map<String, Optional<Employee>> result =  
    employees.stream()  
        .collect(Collectors.groupingBy(  
            Employee::getDepartment,  
            Collectors.maxBy(  
                Comparator.comparingDouble(Employee::getSalary)  
            )  
        ));
```

Result:

```
IT      -> Optional(Employee{id=2, salary=120000})
HR      -> Optional(Employee{id=5, salary=90000})
Finance -> Optional(Employee{id=7, salary=150000})
```



If interviewer wants Map<String, Employee>

</> Java



```
Map<String, Employee> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.collectingAndThen(
                Collectors.maxBy(
                    Comparator.comparingDouble(Employee::getSalary)
                ),
                Optional::get
            )
        ));
```

What to say in interview

"First I group employees by department using `groupingBy()`. Within each department, I use `maxBy()` with a salary comparator to find the employee having the highest salary."

Important variation

If the interviewer asks only for the **maximum salary per department**, not the employee:

</> Java



```
Map<String, Double> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.maxBy(
                Comparator.comparingDouble(Employee::getSalary)
            )
        ))
    .entrySet()
    .stream()
    .collect(Collectors.toMap(
        Map.Entry::getKey,
```

```
        e -> e.getValue().map(Employee::getSalary).orElse(0.0)
    ));
```

A simpler approach for salary only is:

</> Java



```
Map<String, OptionalDouble> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.collectingAndThen(
                Collectors.toList(),
                list -> list.stream()
                    .mapToDouble(Employee::getSalary)
                    .max()
            )
        ));
```

2. Count occurrences of each word

Input

</> Java



```
List<String> words = Arrays.asList(
    "java", "spring", "java", "sql", "spring", "java"
);
```

Solution

</> Java



```
Map<String, Long> frequency =
    words.stream()
        .collect(Collectors.groupingBy(
            Function.identity(),
            Collectors.counting()
        ));
```

Output:

```
java=3  
spring=2  
sql=1
```



Why `Function.identity()` ?

This:

```
</> Java  
  
Function.identity()
```



means:

"Use the element itself as the key."

So:

```
</> Java  
  
groupingBy(Function.identity())
```



is effectively:

```
</> Java  
  
groupingBy(word -> word)
```



Alternative

```
</> Java  
  
Map<String, Long> frequency =  
    words.stream()  
        .collect(Collectors.toMap(  
            Function.identity(),  
            word -> 1L,  
            Long::sum  
        ));
```



Interview point

If interviewer asks:

"Why would you prefer `groupBy()` here?"

Say:

"`groupBy()` naturally represents the grouping-and-counting requirement, while `toMap()` requires an explicit merge function to handle duplicate keys."

3. Find common elements between two lists

Input

`</> Java`



```
List<Integer> list1 = Arrays.asList(1, 2, 3, 4, 5);  
List<Integer> list2 = Arrays.asList(3, 4, 5, 6, 7);
```

Expected:

[3, 4, 5]



Simple Stream solution

`</> Java`



```
List<Integer> common =  
    list1.stream()  
        .filter(list2::contains)  
        .collect(Collectors.toList());
```

This is easy to understand, but there is an important performance issue.

Better solution

Convert the second list to a `Set` :

`</> Java`



```
Set<Integer> set2 = new HashSet<>(list2);  
  
List<Integer> common =  
    list1.stream()
```

```
.filter(set2::contains)
.collect(Collectors.toList());
```

Why is this better?

For a `List` :

```
</> Java

list2.contains(x)
```

is generally $O(n)$.

For a `HashSet` :

```
</> Java

set2.contains(x)
```

is $O(1)$ average case.

Therefore, instead of repeatedly scanning `list2` , we get much faster membership checks.

If duplicate common elements should be removed

```
</> Java

List<Integer> common =
    list1.stream()
        .filter(set2::contains)
        .distinct()
        .collect(Collectors.toList());
```

4. Convert a List into comma-separated String

Input

```
</> Java

List<String> names =
    Arrays.asList("John", "Alex", "David");
```

Expected:

John,Alex,David



Best solution

</> Java



```
String result = names.stream()  
    .collect(Collectors.joining(","));
```

Output:

John,Alex,David



With spaces

</> Java



```
String result = names.stream()  
    .collect(Collectors.joining(", "));
```

Output:

John, Alex, David



With prefix and suffix

</> Java



```
String result = names.stream()  
    .collect(Collectors.joining(", ", "[", "]"));
```

Output:

[John, Alex, David]



★ Important variation: List<Employee> → comma-separated names

If:

</> Java

```
List<Employee> employees;
```



and you want:

John, Alex, David



Use:

</> Java

```
String result = employees.stream()
    .map(Employee::getName)
    .collect(Collectors.joining(", "));
```



The flow is:

```
List<Employee>
  ↓
map(Employee::getName)
  ↓
Stream<String>
  ↓
joining(", ")
  ↓
"John, Alex, David"
```



🔥 Four patterns to remember

Question

Main Stream concept

Maximum salary per department

groupingBy() + maxBy()

Word frequency

groupingBy() + counting()

Question	Main Stream concept
Common elements	<code>filter()</code> + <code>contains()</code>
List → comma-separated String	<code>Collectors.joining()</code>

For your **5-year Java interview preparation**, I'd especially memorize the combinations:

`</>` Java



```
groupingBy() + counting()
groupingBy() + maxBy()
groupingBy() + averagingDouble()
groupingBy() + mapping()
groupingBy() + collectingAndThen()
```

These combinations are much more important than memorizing isolated Stream methods.



ChatGPT can make mistakes. Check important info.